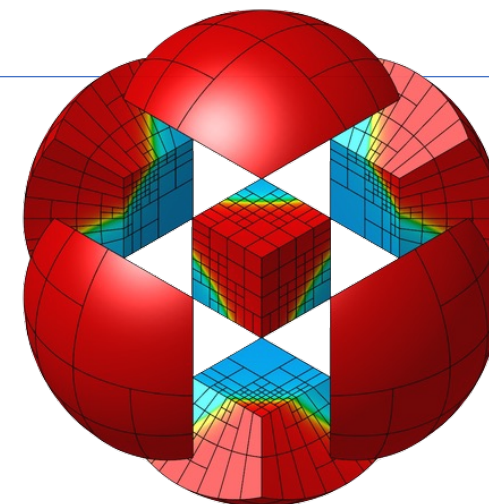


MFEM: Recent Developments

MFEM Community Workshop 2026
September 22-25, 2026, Georgia Tech + Virtual



Veselin Dobrev and the MFEM team

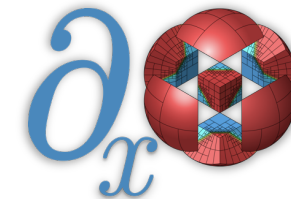
New developments in MFEM

- Two new releases since last year
 - Version 4.9, December 2025
 - Version 4.10, September 2026
- C++17 standard required since v4.9
- New policy for AI-assisted contributions, see [CONTRIBUTING.md](#)

New developments in MFEM

- Discretization improvements

- ∂ FEM: MFEM's Automatic Differentiation (AD) framework for nonlinear finite element operators (talk by **Leonardo Molinari**)



- Based on **Enzyme** or dual numbers at quadrature points
- Added in v4.9; more enhancements coming soon

github.com/EnzymeAD/Enzyme

- Initial support for particle methods

- Classes: **Particle**, **ParticleSet**, and **ParticleVector**
- See miniapps in **miniapps/gslib** and **miniapps/fluids/navier**

- Contact mechanics miniapp using **Tribol**

github.com/LLNL/Tribol

- See miniapps in **miniapps/contact**

New developments in MFEM

- Discretization improvements
 - New integrators for weak Dirichlet B.C. in the hyperbolic framework
 - Classes `BdrHyperbolicDirichletIntegrator`, `BoundaryHyperbolicFlowIntegrator`
 - Support for interior face integration (DG methods) in `ParMixedBilinearForm`, `ParNonlinearForm` and `ParBlockNonlinearForm`
 - Introduced IMEX ODE solvers based on a split-operator framework
 - See examples `ex41` and `ex41p`
 - New method to compute piecewise linear bounds on high-order functions on tensor-product elements

New developments in MFEM

- Discretization improvements
 - Support for complex-valued **mixed** bilinear forms
 - Classes `MixedSesquilinearForm` and `ParMixedSesquilinearForm`
 - Enhanced `GridFunction::ProjectCoefficient` to support selectable projection types
 - `ELEMENT`: nodal interpolation
 - `ELEMENT_L2`: local L2 projection + weighted averaging
 - `GLOBAL_L2`: global L2 projection
 - New **simplex** quadrature rules with symmetric positive weights for triangles (orders 0-25) and tetrahedra (orders 0-20)

New developments in MFEM

- Discretization improvements
 - Support for general 1D Gauss-Jacobi quadrature rules and Stroud conical quadrature rules on triangles and tetrahedra
 - Extended `FindPointsGSLIB` to support point search and interpolation on surface meshes
 - New methods to estimate grid function extrema using piecewise linear bounds plus recursive subdivision, see `GridFunction::GetBounds()`
 - Added `GridFunction` projection methods for trace spaces, i.e., project coefficients on the mesh skeleton

New developments in MFEM

- Discretization improvements
 - New method, `(Par)FiniteElementSpace::GetBoundaryLoopEdgeDofs`, useful for imposing boundary conditions on boundary edge DOFs
 - Added support for coefficient-weighted transfer in `L2ProjectionGridTransfer`
 - Conserves the weighted mass, for example when transferring velocity while conserving density-weighted momentum
 - This is illustrated in the `lor-transfer` and `plor-transfer` miniapps
 - Added support for batched LOR assembly on highly connected meshes
 - Fixed projection on periodic nonconforming meshes and for Nedelec segment elements using the `IntegratedGLL` basis

New developments in MFEM

- Discretization improvements
 - Added support for **INTEGRAL** map type spaces to **QuadratureInterpolator** and **MixedScalarCurlIntegrator**
 - Added the **FiniteElement** methods **GetPhysRangeDim** and **GetPhysCurlDim**,
 - Added new **MapType** values **H_DIV_R1D**, **H_CURL_R1D**, **H_DIV_R2D**, and **H_CURL_R2D**, now used by the **ND_R1D**, **ND_R2D**, **RT_R1D**, and **RT_R2D** elements
 - Added support for selecting the pyramid basis type (Bergot or Fuentes) in **H1_FECollection**, **L2_FECollection**, and the **SetCurvature** methods of **(Par)Mesh**, improving support for high-order meshes with pyramids

New developments in MFEM

- Meshing improvements
 - Introduced NC-patch NURBS meshes
 - Conforming element-wise but allow for nonconforming patch topology
 - This new mesh format supports element spacing formulas for refinement, as well as local refinement factors for a subset of knot vectors
 - New miniapp for interpolating a surface grid of points in 3D using a smooth NURBS surface
 - It can then be sampled at arbitrary resolution while staying close to the original geometry
 - See [miniapps/nurbs/nurbs_surface](#)
 - Improved support for 1D NURBS meshes with variable order, including using the "patches" construct for 1D NURBS meshes

New developments in MFEM

- Meshing improvements
 - Parallel anisotropic refinement of hexahedral meshes is now supported
 - Requires that neighboring hexahedra are not refined in conflicting directions
 - A new **ParMesh** method is added to check for such conflicts, before refinement
 - Added support for nonuniform anisotropic mesh refinement on parallel quad/hex meshes with arbitrary spacing in each direction
 - This enables, in particular, 3:1 refinement in parallel
 - See the new meshing miniapp **pref321**
 - Added support for higher order meshes in **(Par)Mesh::MakeSimplicial**
 - Added support for reading Gmsh meshes in format version 4.1, in both ASCII and binary form, in addition to the previously supported version 2.2

New developments in MFEM

- Meshing improvements

- Added an option to guarantee mesh validity during TMOP-based r-adaptivity, using bounds on the determinant of the mesh transformation Jacobian
 - See `TMOPNewtonSolver::EnsurePositiveDeterminantBound()`
- Added the option to include material interfaces (faces separating elements with different element attributes) as additional boundary elements, for parallel visualization, e.g. with GLVis
 - This is supported by both the `Print` and `PrintAsOne` methods of `ParMesh`
 - See `ParMesh::SetPrintInterfaces()`

New developments in MFEM

- Meshing improvements
 - Added partial assembly (PA) support for TMOP's adaptive limiting
 - Multiple **GridFunctions** and **Coefficients** can be combined to form a composite term
 - Fixed the preservation of element attributes during parallel rebalancing of nonconforming meshes
 - The TMOP kernel hierarchy has been restructured to reduce compilation time

New developments in MFEM

- Linear and nonlinear solvers
 - Added **FilteredSolver**: a base class for solvers with filtering. It handles cases where a solver performs well except in small subspaces, by adding a filtering step formulated as a subspace correction
 - Added **AMGFSolver**: a derived class of **FilteredSolver**, specialized for AMG with Filtering (AMGF), providing robust preconditioning for linear systems arising in constrained optimization problems such as frictionless contact

New developments in MFEM

- Linear and nonlinear solvers
 - Added support for MPI-distributed solvers and preconditioners from the Ginkgo library
 - See the new parallel example in `examples/ginkgo/ex1p`
 - Added `SuperLUSolver::SetDeviceOffload` to enable or disable device offload when using a GPU-enabled SuperLU_DIST installation
 - The PETSc interface now requires PETSc to be built with hypre support
 - Changed the default solver parameters of `HyprePCG` to `tol=1e-6` and `max_iter=1000`
 - This matches the default parameters in hypre 3.0

New developments in MFEM

- Linear and nonlinear solvers
 - Added the methods `GetResiduals` and `GetFinalAbsResidualNorm` to `HyprePCG`, `HypreGMRES`, and `HypreFGMRES`, to access the residual vector `r` and compute `|r|_p` from it
 - Note that the latter computes `|r|_p` from `r` instead of returning a cached value like the relative `GetFinalResidualNorm`
 - Added new class `MultiVector`, representing an array of `Vectors` of different sizes, where each `Vector` can be allocated independently
 - Associated methods in class `Operator`: `MultMV`, `MultTransposeMV`, and `GetGradientMV` use the `MultiVector` objects for input and/or output parameters

New developments in MFEM

- Linear and nonlinear solvers
 - Added support for trace spaces in `PRefinementTransferOperator`
 - Used in p-refinement multigrid methods for problems posed on trace spaces
 - See e.g. the DPG miniapps
 - Extended `TimeDependentOperator::ImplicitSolve` to optionally solve for the stage-state, u , instead of the stage-slope, du/dt , as specified by the new `ImplicitVariableType` setting
 - Support varies by ODE solver and can be queried with `ODESolver::SupportsImplicitVariableType()`
 - See Examples 9 and 16

New developments in MFEM

- GPU computing

- Added the `gpu`, `raja-gpu`, and `ceed-gpu` backend aliases/shortcuts which automatically select between CUDA or HIP
- Added the option to enable GPU-aware MPI in MFEM using the environment variable `MFEM_GPU_AWARE_MPI` set to any value
 - Alternative to calling `Device::SetGPUAwareMPI(true)`
- Implemented a GPU-accelerated matrix-free AMR derefinement `GridFunction` update operator
 - Supports mixed geometry meshes and variable order spaces
 - Default derefinement operator constructed by `(Par)FiniteElementSpace::Update`
 - The operator requires the finite element space to be nonconforming

New developments in MFEM

- GPU computing
 - Introduced `MFEM_FOREACH_THREAD_DIRECT`, which directly maps loop tasks to GPU threads, assigning one task per thread
 - The function `Vector::SetSubVector(const Array<int> &, const real_t)` now executes on device if either the vector or the array have the device flag set
 - This is most often used for setting constant essential BCs
 - A new method, `SetSubVectorHost`, has been added for cases where host execution is always needed (e.g. when the DOFs array is small)
 - Added GPU support in `GradientGridFunction` and `InnerProduct Coefficient` classes by implementing their `Project` methods

New developments in MFEM

- GPU computing
 - Added new method: `GridFunction::GetGradients`, with GPU support, for computing the gradients of a `GridFunction` on all elements
 - The CUDA-specific names used by some of the unit tests like `cunit_tests` and `pcunit_tests` were replaced by names using `gpu` instead of `c` (short for CUDA) or `cuda`
 - These tests automatically run the CUDA/HIP tests based on the MFEM build configuration

New developments in MFEM

- GPU computing
 - Added GPU-enabled partial assembly for **simplicial** Bernstein H1 basis based on ragged tensor algorithms for **mass** and **diffusion** integrators
 - Added NVIDIA cuDSS library interface
 - Implementation examples have been added to **ex1** and **ex1p**
 - See <https://developer.nvidia.com/cudss> for details
- Added device support to the **ParticleSet** and **ParticleVector** classes

New developments in MFEM

- GPU computing
 - Improved GPU kernels for many existing PA integrators:
 - PA diagonal kernels for **VectorMassIntegrator** (shared-memory specializations) and **ElasticityIntegrator** (no scratch Q-vector)
 - PA for **VectorDivergenceIntegrator** with shared-memory kernels, kernel registration, and transpose support
 - PA gradient and diagonal for **VectorConvectionNLFIntegrator** (**AssembleGradPA**, **AddMultGradPA**, **AssembleGradDiagonalPA**)
 - PA of **VectorFEMassIntegrator** now uses kernel specialization dispatch

New developments in MFEM

- GPU computing
 - Added device assembly support for a number of electromagnetic integrators:
 - 3D H(curl) `VectorFEDomainLFIntegrator`
 - `MixedScalarWeakGradientIntegrator`
 - `MixedDotProductIntegrator`
 - `MixedScalarWeakCrossProductIntegrator`
 - `MixedScalarCrossProductIntegrator`
 - H1- $\rightarrow$ RT `MixedVectorGradientIntegrator`

New developments in MFEM

- GPU computing
 - Added support for device partial assembly for the `CurlInterpolator`, in 2D/3D:
 - 2D H1 (out-of-plane) to RT (in-plane)
 - 2D ND (in-plane) to Integral L2 (out-of-plane)
 - 3D ND to RT
 - Added support for specifying GPU kernel **launch bounds** for the native and RAJA GPU backends, via a new optional template parameter of the `mfem::forall_2D` and `forall_3D` kernel wrappers
 - Extended the GPU-accelerated hybridization support to nonconforming meshes

New developments in MFEM

- GPU computing
 - Added device support to class `GroupCommunicator`:
 - Methods that take the data to be communicated as raw pointers continue to execute on host and expect the raw pointer to be a host pointer
 - Methods that take the data to be communicated as `Array<T>&` will perform the operations on the configured `mfem::Device` and will use GPU-aware MPI, if enabled for the device
 - Not all cases are supported on device; in such cases, execution will fall back to host

New developments in MFEM

- New and updated examples and miniapps
 - Added the `miniapps/fluids` directory and moved the previous Navier and the new incompressible Schrödinger flow miniapps into it
 - Introduced the new Incompressible Schrödinger Flow (ISF) miniapp, which models inviscid fluid dynamics by solving the linear Schrödinger equation, leveraging the hydrodynamical analogy to quantum mechanics

New developments in MFEM

- New and updated examples and miniapps
 - New particle-related miniapps:
 - New transient Navier-Stokes fluid-particles solver NavierParticles in `miniapps/fluids/navier/navier_particles`, for modeling tracer particles in fluid flow, demonstrating use of the new `ParticleSet` class
 - New Navier miniapp, `miniapps/fluids/navier/navier_bifurcation`, showing the use of NavierParticles in a 2D bifurcating channel flow
 - New `FindPointsGSLIB` miniapp, `miniapps/gslib/particles_redist`, showing parallel-redistribution of particle data between MPI ranks
 - Particle visualization features in `common/particles_extras` for viewing particle locations and trajectories (`ParticleTrajectories`) using GLVis

New developments in MFEM

- New and updated examples and miniapps
 - Added a new miniapp ([meshing/mesh-bounding-boxes](#)) that computes the bounding boxes for each element of a given mesh, and the bounds on the determinant of the Jacobian of the transformation
 - Added a new miniapp ([tools/gridfunction-bounds](#)) to compute piecewise linear bounds on a given high-order grid function
 - Added a new miniapp ([electromagnetics/lorentz](#)) which computes the trajectory of a charged particle, subject to Lorentz forces, in electrostatic and/or magnetostatic fields as computed by the [volta](#) or [tesla](#) miniapps

New developments in MFEM

- New and updated examples and miniapps
 - Added miniapps to demonstrate an implementation of the absolute-value l1-Jacobi preconditioners in partially assembled operators
 - This includes Multigrid wrapper to demonstrate the effectiveness of these Jacobi-type operators as smoothers
 - See the [miniapps/diag-smoothers](#) directory
 - Updated the [mtop](#) miniapp with a GPU enabled forward and adjoint solver for isotropic linear elasticity

New developments in MFEM

- New and updated examples and miniapps
 - Added a new Particle-In-Cell (PIC) miniapp in `miniapps/plasma/pic`, performing electrostatic PIC simulation of charged particles in 2D or 3D using the `ParticleSet` capability
 - Each time step deposits the particle charges on the grid, solves a Poisson problem for the electric potential, interpolates the electric field, and pushes the particles with a leap-frog scheme in a periodic domain
 - Added new NURBS miniapps:
 - `nurbs_ex10` and `nurbs_ex10p`, NURBS versions of Example 10 (time-dependent nonlinear elasticity) based on the new NURBS projection support
 - `nurbs_mesh_info`, which prints information about NURBS meshes

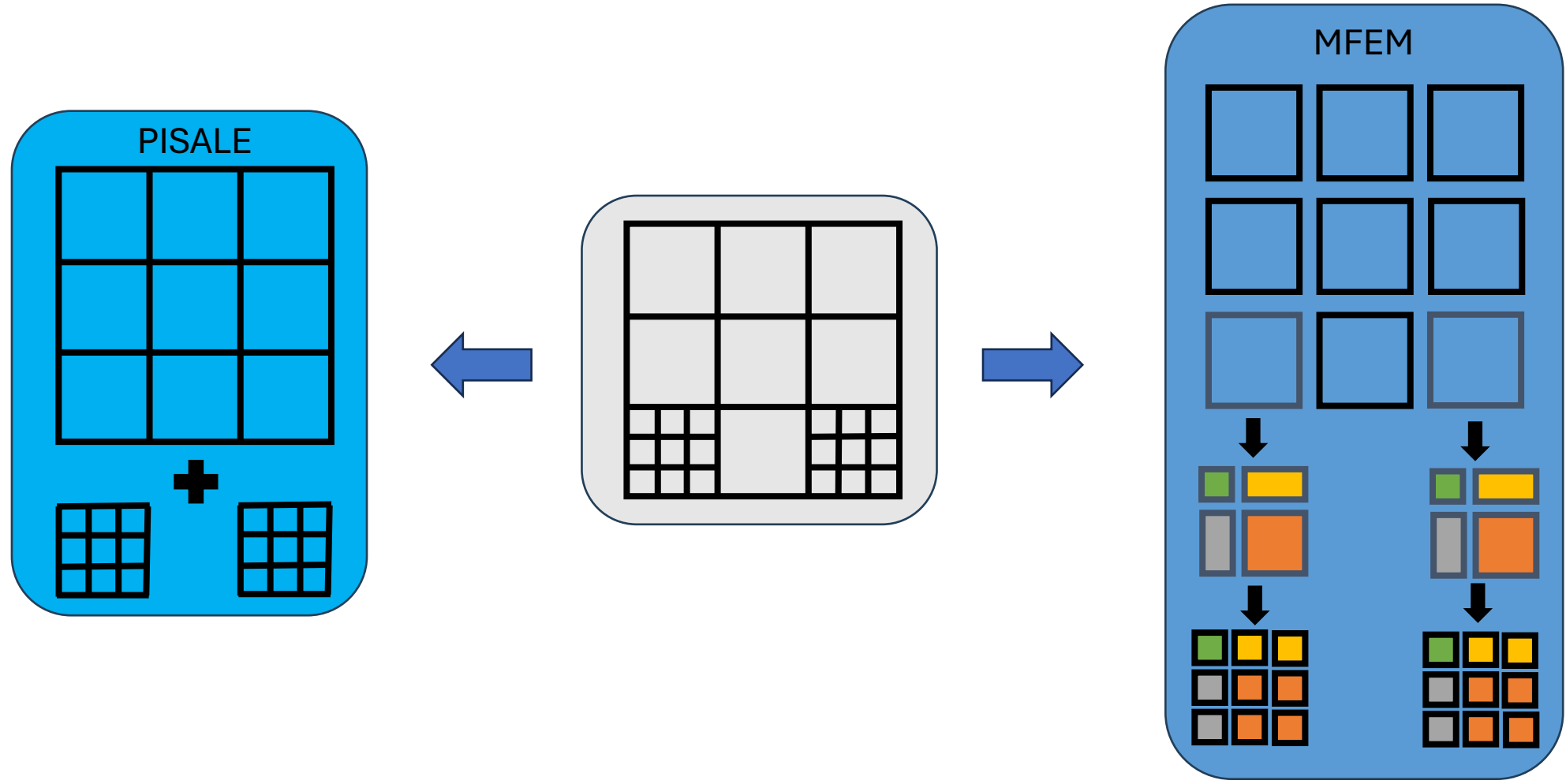
New developments in MFEM

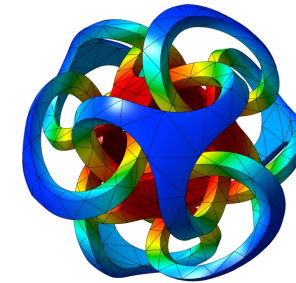
- New and updated examples and miniapps
 - Added a new miniapp (`tools/compare-dc`) that loads fields saved via `DataCollection` classes and computes the norm of their difference, optionally checking it against a given tolerance
 - Currently, only the `VisItDataCollection` class is supported
 - The Lorentz miniapp (in `miniapps/electromagnetics`) was updated to leverage the `ParticleSet` capability, including its new device support
 - Added `(Complex)PRefinementMultigrid` solver option in the DPG miniapps

New developments in MFEM

- And many more ...

Highlight: Support 3-to-1 patch ratios by repeated non-uniform refinements





New developments in GLVis

- One new release since last year: v4.5, February 2026
- Requires new MFEM and C++17
- Added visualization of complex data for grid functions from MFEM's class `(Par)ComplexGridFunction`
 - The complex values are represented by their magnitude, phase, real part or imaginary part, which can be cycled with the `q` key
 - Additionally, complex functions can be animated by an added harmonic phase. The phase frequency can be increased/decreased with keys `;`/`:`

New developments in GLVis

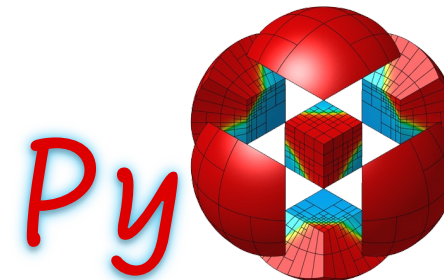
- Added the option to choose palettes by name using the command line argument `-pname` or `--palette-name`
 - This option is also available in scripts and streams using the keyword `palette_name`
- Color palettes defined in external files
- Headless (no GUI) visualization, relying on the EGL (Linux) or CGL (macOS) interface
- Generalized and unified processing of stream commands for `glvis-js` to support all commands

New developments in GLVis

- Added visualization of the finite element DOF numbering in 2D scalar mode, with the **n/N** keys that now cycles through: None → Elements → Edges → Vertices → DOFs
 - In parallel, the default numbering is the "local" one for each rank. This can be switched to the "global" parallel numbering with **Alt+n** (the former numbering method)
- And more ...

MFEM events and resources

- FEM@LLNL seminar series, online, sign-up at mfem.org/seminar
- PyMFEM: github.com/mfem/PyMFEM





Thank you!